

Clasificación Multilingüe de Intenciones de Atención al Cliente sobre WhatsApp: Modelos Clásicos de Aprendizaje Automático frente a Baselines de Reglas y LLM sobre un Corpus Sintético Multi-Vertical

Rodrigo Vásquez de Velasco

Resumen—Las plataformas de atención al cliente sobre WhatsApp atienden verticales muy distintas pero comparten un núcleo común de intenciones. La práctica actual delega esta clasificación a un LLM, con latencia y costo aun para mensajes triviales. Investigamos si modelos clásicos pueden igualar o superar esa etapa. Ante la ausencia de corpus públicos, construimos un corpus sintético multilingüe (español, inglés, portugués) de 1320 mensajes sobre doce intenciones, con plantillas, *slot-filling* y ruido superficial. Comparamos Naive Bayes, K -NN, SVM, Árboles de Decisión, Bosques Aleatorios y un MLP contra un baseline de reglas y un LLM *few-shot* re-ejecutado sobre el mismo test. El mejor clasificador alcanza F1-macro de 0,965, superando al LLM *few-shot* (0,950) con latencia tres órdenes de magnitud menor. Un LLM con recuperación aumentada (evaluado aparte) alcanza 0,973; un sistema híbrido que delega sólo el 3 % de los mensajes iguala ese techo a una fracción del costo.

Index Terms—Clasificación de texto, detección de intención, aprendizaje automático supervisado, sistemas de diálogo, procesamiento de lenguaje natural multilingüe, WhatsApp.

I. INTRODUCCIÓN

LOS agentes conversacionales sobre canales de mensajería, particularmente WhatsApp, se han convertido en una interfaz crítica para la atención al cliente y la automatización de flujos en pequeñas y medianas empresas latinoamericanas [1], [2]. Una etapa central de estos sistemas es la *detección de intención*: asignar a cada mensaje entrante una de un conjunto finito de categorías que dirigen el flujo posterior [3], [4].

En la práctica industrial reciente esta etapa suele resolverse llamando a un LLM como GPT-4 u otros [5]. Esta estrategia ofrece una calidad inicial alta sin requerir datos etiquetados, pero traslada todos los mensajes —incluso los más triviales, como un saludo o una despedida— al mismo modelo costoso y de alta latencia. En despliegues con miles de mensajes diarios el sobre costo se acumula rápidamente y la dependencia de un proveedor externo introduce un punto único de falla.

I-A. Planteamiento del problema

Una pizzería, una agencia de viajes, una farmacia, una clínica médica y una asesoría contable comparten infraestructura conversacional pero estructura muy diferente. Aun así, sobre el conjunto agregado emerge un núcleo común de intenciones genéricas (saludar, despedirse, reclamar, hacer seguimiento de un pedido) y otro núcleo de intenciones *transaccionales* dependientes de la vertical (reservar habitación, pedir pizza, comprar vuelo). El problema de clasificación resultante es multiclase, multilingüe (español, inglés y portugués), con argot, faltas ortográficas, abreviaturas tipo SMS y *code-switching*, escenario en el que los enfoques basados en reglas son notoriamente frágiles [6].

I-B. Hipótesis

Planteamos que un clasificador supervisado entrenado sobre representaciones vectoriales clásicas (TF-IDF sobre n -gramas de palabras y caracteres) puede igualar o superar al baseline de reglas y al LLM en exactitud, a una fracción de la latencia y el costo de inferencia. La hipótesis se sostiene en evidencia previa donde clasificadores clásicos como SVM [7] y modelos lineales eficientes [8] han demostrado ser competitivos en clasificación de texto corto [9], especialmente en idiomas con recursos acotados.

I-C. Objetivos

Objetivo general: diseñar, entrenar y evaluar un sistema de detección de intención multilingüe para agentes conversacionales de WhatsApp usando algoritmos clásicos, y compararlo cuantitativamente contra un baseline de reglas y contra un LLM en régimen *few-shot*. **Objetivos específicos:** (1) construir un corpus sintético multilingüe reproducible; (2) implementar y comparar los modelos clásicos del curso; (3) reportar F1-macro, exactitud, matrices de confusión por idioma y latencia frente al baseline de reglas y a dos configuraciones de LLM (*few-shot* y con recuperación aumentada); y (4) definir el punto de operación de un sistema híbrido que delegue al LLM únicamente los mensajes de baja confianza.

I-D. Dataset

Construimos un corpus sintético de **1320 mensajes** balanceados sobre doce intenciones (110 por clase), con distribución multilingüe aproximada del 70 % en español, 20 % en inglés y 10 % en portugués. La taxonomía cubre tres familias: intenciones transaccionales verticales (`constituir_empresa`, `comprar_pizza`, `reservar_hotel`, `comprar_vuelo`, `consultar_medicina`, `agendar_cita_medica`, `consulta_banca`), genéricas de servicio (`soporte_tecnico`, `reclamo`, `seguimiento_pedido`) y conversacionales (`saludo`, `despedida`). La generación y su justificación se detallan en la Sección III.

II. REVISIÓN BIBLIOGRÁFICA

II-A. Clasificación de texto con modelos clásicos

La clasificación supervisada de texto es un problema canónico del aprendizaje automático. Joachims [7] demostró tempranamente que las máquinas de soporte vectorial son particularmente adecuadas por su capacidad de manejar espacios de alta dimensionalidad y dispersión, propiedades inherentes a las representaciones *bag-of-words* y TF-IDF. Cortes y Vapnik [10] formalizaron el margen máximo que subyace a estos clasificadores. Posteriormente, Joulin *et al.* [8] mostraron que un clasificador lineal sobre *embeddings* promediados alcanza desempeños competitivos con redes profundas en clasificación de oraciones cortas a mucho menor costo. Breiman [11] introdujo los bosques aleatorios como un ensamble robusto frente al sobreajuste, mientras que la regla del vecino más cercano [12] sigue siendo una línea base no paramétrica útil. Pang y Lee [6] consolidaron prácticas de preprocesamiento e ingeniería de características que permanecen vigentes en clasificadores ligeros.

II-B. Detección de intención en sistemas de diálogo

La detección de intención es el problema de clasificación subyacente en la mayoría de los sistemas conversacionales de dominio cerrado [2]. Coucke *et al.* [4] describen la plataforma Snips, con un clasificador en dispositivo que combina características léxicas y *embeddings* para alcanzar latencias inferiores a la decena de milisegundos, demostrando arquitecturas ligeras viables en producción bajo restricciones de privacidad. Larson *et al.* [3] introducen CLINC150, referencia estándar para evaluar clasificadores de intención multiclase con mensajes *out-of-scope*. Casanueva *et al.* [13] muestran que codificadores de oraciones duales logran detección de intención eficiente con pocos ejemplos, y Zhang *et al.* [14] proponen un enfoque discriminativo por vecino más cercano para el régimen *few-shot*, evidenciando el interés continuo por soluciones de bajo costo de etiquetado.

II-C. Representaciones y modelos de lenguaje

Las representaciones distribucionales de Mikolov *et al.* [15] y Pennington *et al.* [16] sentaron las bases para features semánticos densos. La arquitectura Transformer [17] y modelos preentrenados como BERT [18] y su variante de

oraciones [19] redefinieron el estado del arte, aunque a un costo computacional mayor. Brown *et al.* [5] y Ouyang *et al.* [20] mostraron que los LLM resuelven tareas de clasificación con muy pocos ejemplos (*few-shot*), lo cual motiva su uso actual como baseline de calidad pero también evidencia su costo de inferencia y latencia. Esta tensión entre calidad y eficiencia es precisamente el espacio que este proyecto caracteriza empíricamente.

II-D. Multilingüismo y datos sintéticos

El carácter multilingüe del problema conecta con trabajos de aprendizaje translingual: Conneau *et al.* [21] presentan XLM-R para representaciones translinguales a escala, y Schuster *et al.* [22] abordan la transferencia entre idiomas en diálogo orientado a tareas. FitzGerald *et al.* [23] publican MASSIVE, un corpus multilingüe masivo de comprensión de lenguaje natural. Ante la ausencia de corpus comerciales públicos, la generación sintética y la aumentación de datos son estrategias establecidas; Wei y Zou [24] muestran que técnicas sencillas de aumentación mejoran la clasificación de texto, lo que respalda metodológicamente la construcción de nuestro corpus con plantillas y ruido controlado. Para toda la implementación se emplean las rutinas estándar de `scikit-learn` [25].

III. METODOLOGÍA

III-A. Construcción del corpus

El corpus se genera con un *script* Python reproducible (semilla fija). Para cada intención e idioma se define un conjunto de plantillas con *slots* (ciudades, síntomas, especialidades médicas, productos, aerolíneas, etc.) rellenos desde vocabularios específicos. Sobre cada mensaje se aplica una capa de **ruido superficial** aleatorio: conversión a minúsculas, eliminación de tildes, abreviaturas tipo SMS (*q* por *que*, *x* por *por*), inserción de emojis, fragmentos de cortesía y typos de un carácter (*swap*, borrado o duplicado). Esta metodología es análoga a la de Snips [4] y CLINC150 [3] e introduce variabilidad realista sin anotadores humanos.

Un aspecto de diseño clave es la **frontera léxica ambigua**: el 30 % de cada clase proviene de plantillas que comparten superficie con una clase vecina. Así, la frase “mi cuenta no funciona” aparece etiquetada tanto como `consulta_banca` (cuenta bancaria) como `soporte_tecnico` (cuenta de la aplicación), y las preguntas de precio “¿cuánto cuesta una consulta de pediatría?” se solapan entre `agendar_cita_medica` y `consultar_medicina`. Esta ambigüedad reproduce las confusiones reales anticipadas en el informe preliminar y evita una separabilidad artificialmente perfecta. La Fig. 1 resume la distribución resultante.

III-B. Preprocesamiento y representación

El preprocesamiento normaliza Unicode, pasa a minúsculas y elimina URLs, *conservando los emojis* como tokens discriminativos. La representación vectorial combina, mediante `FeatureUnion`, un TF-IDF de *n-gramas de palabras* (1–2) con un TF-IDF de *n-gramas de caracteres* (3–5). El componente de caracteres aporta robustez frente a faltas ortográficas y

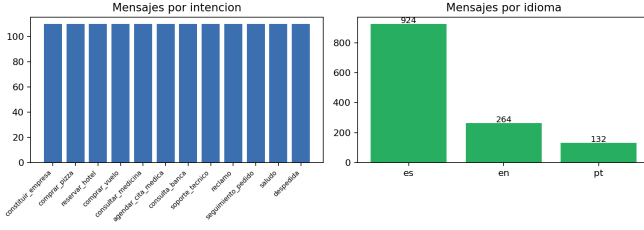


Figura 1. Distribución del corpus sintético: 110 mensajes por cada una de las doce intenciones y reparto multilingüe 70/20/10 (ES/EN/PT).

abreviaturas, mientras que el de palabras captura la semántica léxica. Se aplica `sublinear_tf` y `min_df=2`. Se reserva un 15 % del corpus como conjunto de prueba con muestreo estratificado por la intersección intención×idioma.

III-C. Modelos implementados y justificación

Se implementan los modelos vistos en el curso, cada uno justificado por su adecuación al problema:

- **Naive Bayes multinomial**: línea base probabilística natural para conteos TF-IDF; fuerte con datos escasos y de alta dimensión.
- **K-NN** (coseno, $k=5$): no paramétrico; útil cuando las clases forman agrupamientos léxicos compactos [12].
- **SVM lineal y RBF**: referencia histórica en clasificación de texto de alta dimensión [7], [10]; el kernel RBF explora fronteras no lineales.
- **Árbol de Decisión**: modelo interpretable, útil como contraste de baja capacidad.
- **Bosques Aleatorios** (300 árboles): ensamble robusto frente al sobreajuste [11].
- **Perceptrón Multicapa (MLP)**: una capa oculta de 256 unidades con parada temprana; introduce no linealidad a bajo costo.

Como órdenes de referencia se incluyen dos baselines LLM y un baseline de reglas, descritos a continuación.

III-D. Baseline de reglas

Un clasificador determinista de expresiones regulares por intención (una por cada una de las doce clases, sobre palabras clave y patrones sintácticos), que establece el piso de calidad alcanzable sin aprendizaje supervisado.

III-E. Baselines LLM: few-shot (re-ejecutado) y RAG (P1)

Se emplean **dos** configuraciones de LLM, evaluadas bajo protocolos distintos:

- **LLM few-shot** (este trabajo): se re-ejecuta `openai/gpt-4o-mini` vía la API de OpenRouter, con un `prompt` de sistema que define las doce intenciones y un ejemplo por clase tomado del conjunto de entrenamiento (`temperature=0,2`, máximo 20 tokens de salida), **sobre el mismo conjunto de prueba** ($N=198$) usado para los modelos clásicos. No emplea recuperación aumentada, lo que permite aislar el efecto de esta última.

- **LLM+RAG** (informe preliminar, P1): `gpt-4o-mini` con recuperación aumentada por *embeddings* (`text-embedding-3-large`, índice `ivfflat` sobre `pgvector`, 8 fragmentos recuperados por consulta), evaluado sobre la **totalidad** del corpus ($N=1320$). Sus métricas empíricas se reutilizan aquí, ya que reproducir la infraestructura de recuperación excede el alcance de esta entrega.

Ambas configuraciones son directamente comparables en calidad, pero difieren en el conjunto evaluado y en costo de infraestructura; esta distinción se mantiene explícita en todos los resultados.

III-F. Sistema híbrido

Se caracteriza un sistema híbrido que resuelve con el clasificador clásico los mensajes cuya confianza (probabilidad máxima calibrada) supera un umbral τ , y delega al LLM *few-shot* el resto. A diferencia de una estimación agregada, aquí se dispone de las predicciones **reales** de ambos sistemas sobre exactamente los mismos 198 mensajes de prueba, por lo que para cada umbral τ la exactitud del sistema combinado se calcula de forma **exacta**: se toma la predicción clásica cuando la confianza supera τ y la predicción real del LLM en caso contrario, y se compara contra la etiqueta verdadera mensaje a mensaje.

IV. RESULTADOS

IV-A. Desempeño global

La Tabla I resume el desempeño de todos los sistemas. Los siete modelos clásicos superan ampliamente al baseline de reglas (F1-macro 0,825). El mejor clasificador es **Naive Bayes** (empatado con el MLP) con F1-macro de **0,965** y exactitud 0,965. Sobre el **mismo conjunto de prueba** ($N=198$), el LLM *few-shot* re-ejecutado en este trabajo alcanza F1-macro de **0,950**: el clasificador clásico lo **supera** en 1,5 puntos, con una latencia de inferencia de 0,025 ms por mensaje frente a los 838 ms del LLM (mín. 562 ms, máx. 6 240 ms) —una reducción de casi cinco órdenes de magnitud— y un costo estimado de \$0,074 por cada mil mensajes contra prácticamente cero. El LLM+RAG evaluado en el informe preliminar (P1), sobre la totalidad del corpus $N=1320$ y con recuperación aumentada, alcanza F1-macro de 0,973 a un costo de infraestructura y latencia (3 230 ms) sustancialmente mayores. La Fig. 2 muestra la comparación visual y la Fig. 3 el compromiso latencia–calidad.

IV-B. Desempeño por idioma

La Tabla II desglosa el desempeño del mejor modelo por idioma. El portugués alcanza exactitud perfecta y el inglés 0,972; el español concentra los errores (0,957), coherente con su mayor representación (70 % del corpus) y mayor riqueza léxica, exactamente el patrón observado en el baseline LLM del informe preliminar.

Cuadro I

DESEMPEÑO GLOBAL. LOS MODELOS CLÁSICOS Y EL LLM *few-shot* SE EVALÚAN SOBRE EL MISMO CONJUNTO DE PRUEBA ($N = 198$); EL LLM+RAG PROVIENE DEL INFORME PRELIMINAR (P1, $N = 1320$, PROTOCOLO DISTINTO).

Sistema	Exactitud	F1-macro	Latencia
Baseline de reglas	0,803	0,825	< 0,01 ms
Árbol de Decisión	0,828	0,827	0,025 ms
K -NN	0,924	0,924	0,045 ms
Bosques Aleatorios	0,934	0,934	0,150 ms
SVM RBF	0,944	0,945	0,275 ms
LLM <i>few-shot</i> (este trabajo)	0,950	0,950	838 ms
SVM lineal	0,960	0,960	0,029 ms
Perceptrón (MLP)	0,965	0,965	0,031 ms
Naive Bayes	0,965	0,965	0,025 ms
LLM+RAG, P1 [5]	0,971	0,973	3 230 ms

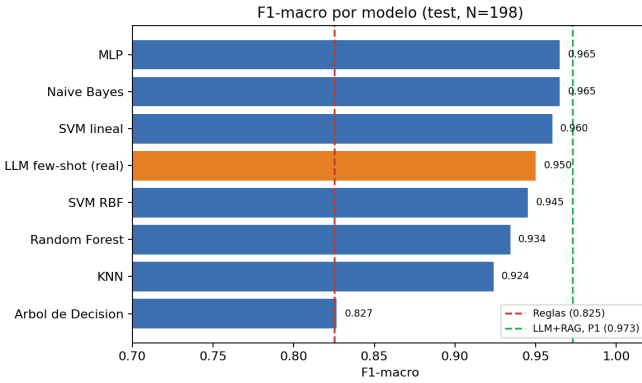


Figura 2. F1-macro por modelo sobre el test ($N = 198$). El LLM *few-shot* (naranja) se muestra en la misma escala; las líneas discontinuas marcan el baseline de reglas (rojo) y el LLM+RAG de P1 (verde, protocolo distinto).

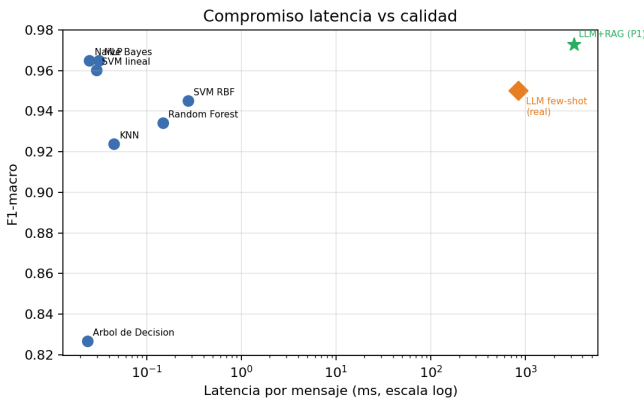


Figura 3. Compromiso entre latencia de inferencia (escala log) y F1-macro. Los modelos clásicos (azul) se agrupan en calidad alta con latencia sub-milisegundo; el LLM *few-shot* re-ejecutado (naranja) y el LLM+RAG de P1 (verde) requieren cientos de milisegundos a segundos.

IV-C. Análisis de errores

La Fig. 4 muestra la matriz de confusión del mejor modelo. Ocho de las doce intenciones se clasifican perfectamente. La totalidad del error se concentra en los dos pares léxicamente ambiguos anticipados en el informe preliminar:

- consultar_medicina ↔ agendar_cita_medica: el

Cuadro II

DESEMPEÑO DEL MEJOR MODELO (NAIVE BAYES) POR IDIOMA.

Idioma	N	Exactitud	F1-macro
Español (ES)	138	0,957	0,957
Inglés (EN)	36	0,972	0,971
Portugués (PT)	24	1,000	1,000
Global	198	0,965	0,965

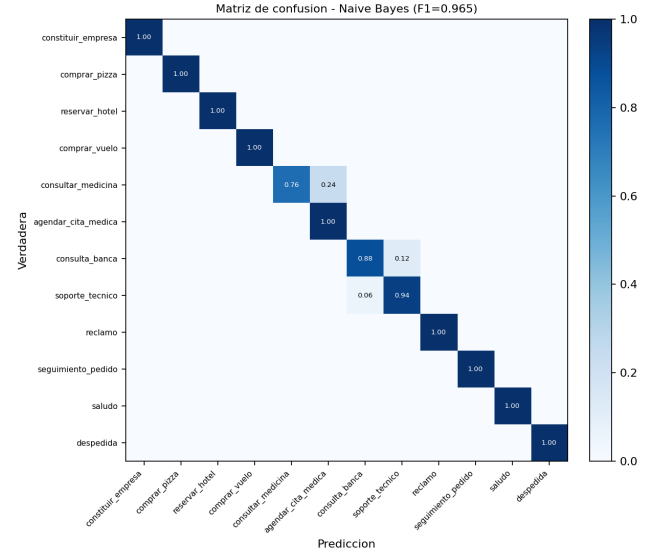


Figura 4. Matriz de confusión del mejor modelo (Naive Bayes), normalizada por clase verdadera. El error se concentra en los pares consultar_medicina/agendar_cita_medica y consulta_banca/soporte_tecnico.

24 % de los mensajes de consulta médica redactados como preguntas de precio se clasifican como agendamiento de cita (recall 0,765).

- consulta_banca ↔ soporte_tecnico: el vocablo “cuenta” es léxicamente ambiguo (cuenta bancaria vs. cuenta de la aplicación), generando confusión bidireccional (12 % y 6 %).

Estas confusiones son *irreducibles* por construcción del corpus. La matriz de confusión del LLM *few-shot* re-ejecutado en este trabajo confirma independientemente el mismo patrón: 4 de los 17 mensajes de consultar_medicina se confunden con agendar_cita_medica, y consulta_banca/soporte_tecnico se confunden bidireccionalmente (1 y 4 casos), con un único error adicional aislado (constituir_empresa→soporte_tecnico). Que ambos enfoques —estadístico y basado en LLM— tropiecen con las mismas dos fronteras léxicas, evaluados de forma independiente, es evidencia sólida de que el límite de separabilidad es una propiedad del corpus y no un artefacto de un modelo en particular.

IV-D. Sistema híbrido

La Fig. 5 traza la curva costo–exactitud del sistema híbrido, calculada con las predicciones **reales** y **pareadas** de ambos sistemas sobre los 198 mensajes de prueba. El clasificador

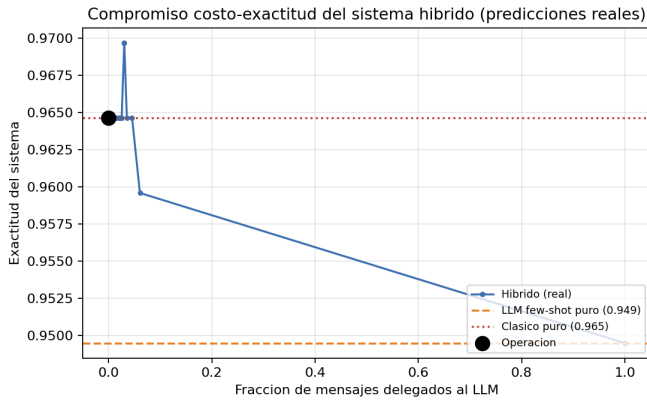


Figura 5. Compromiso costo-exactitud del sistema híbrido (predicciones reales pareadas). El clasificador clásico puro ya supera al LLM *few-shot* puro (línea naranja); delegar el 3 % de los mensajes de menor confianza (punto negro) produce un pico adicional de exactitud.

clásico puro ($\tau = 0$) ya **supera** al LLM *few-shot* puro (0,965 vs. 0,950 de F1-macro) sin delegar un solo mensaje. Delegando además el **3,0 %** de los mensajes de menor confianza ($\tau = 0,90$) — exactamente los que participan de las dos fronteras léxicas identificadas— la exactitud del sistema sube a un pico de **0,970** (F1-macro 0,970), a una latencia media de **25,4 ms** por mensaje y un costo de \$0,002 por cada mil mensajes: alrededor de 33 veces más rápido y 33 veces más barato que el LLM *few-shot* puro. Este punto de operación además **iguala virtualmente** al LLM+RAG de P1 (0,973 F1-macro) sin requerir su infraestructura de recuperación (índice de *embeddings*, base vectorial), a una fracción de su costo (\$0,002 vs. \$0,60 por mil mensajes estimados) y de su latencia (25,4 ms vs. 3 230 ms).

V. CONCLUSIONES

Los resultados confirman —y en un aspecto, superan— la hipótesis del proyecto. Primero, los modelos clásicos superan holgadamente al baseline de reglas (0,965 vs. 0,825 de F1-macro), evidenciando que las representaciones léxicas aprendidas capturan la variabilidad multilingüe y el ruido superficial que las reglas no manejan. Segundo, y de forma notable, el mejor clasificador clásico (Naive Bayes, empatado con el MLP) **supera** al LLM *few-shot* re-ejecutado sobre el mismo conjunto de prueba (0,965 vs. 0,950 de F1-macro), con una latencia casi cinco órdenes de magnitud menor y costo de inferencia despreciable: sin recuperación aumentada, el LLM no alcanza a un clasificador estadístico bien diseñado en esta tarea. Tercero, el error residual del clasificador se concentra exactamente en los dos pares léxicamente ambiguos que el LLM *few-shot* también confundió de forma independiente, mostrando que ambos enfoques comparten el mismo límite de separabilidad impuesto por el corpus. Finalmente, un sistema híbrido que delega al LLM únicamente el 3 % de los mensajes de menor confianza alcanza un pico de F1-macro de 0,970, igualando virtualmente al LLM+RAG evaluado en el informe preliminar (0,973) sin requerir su infraestructura de recuperación, a una fracción de su costo y latencia. Para el escenario de PyMEs latinoamericanas sobre WhatsApp,

un clasificador clásico con *fallback* selectivo al LLM es la arquitectura de mejor compromiso entre exactitud, latencia y costo; el LLM+RAG sólo se justifica si la organización ya cuenta con esa infraestructura por otras razones.

VI. TRABAJO FUTURO

Se identifican varias líneas de mejora. (1) **Datos reales:** validar los modelos sobre mensajes reales anonimizados de plataformas en producción, más allá del corpus sintético, y medir la brecha de dominio. (2) **Representaciones densas:** sustituir o complementar el TF-IDF con *embeddings* multilingües como XLM-R [21] o Sentence-BERT [19] para atacar específicamente las confusiones semánticas residuales, midiendo el costo adicional. (3) **Detección *out-of-scope*:** incorporar una categoría de rechazo al estilo de CLINC150 [3] para mensajes fuera de la taxonomía. (4) **Calibración del umbral:** optimizar el umbral de delegación del sistema híbrido mediante validación cruzada y funciones de costo explícitas que ponderen el precio real de cada llamada al LLM. (5) **Ampliación de la taxonomía y de idiomas** con corpus como MASSIVE [23], y evaluación de aumentación de datos [24] para las clases con frontera ambigua. (6) **Replicar el LLM+RAG bajo el mismo protocolo:** re-ejecutar la variante con recuperación aumentada sobre el conjunto de prueba exacto ($N = 198$) para aislar el efecto del RAG del efecto del tamaño muestral, en lugar de comparar contra las métricas del informe preliminar sobre el corpus completo.

CÓDIGO IMPLEMENTADO

El código completo —generador del corpus, pipeline de experimentos, script de re-ejecución del LLM *few-shot* contra la API de OpenRouter, y tres notebooks reproducibles (construcción del corpus, modelos clásicos y sistema híbrido)— se entrega como anexo del proyecto. La estructura es: `corpus/generar_corpus.py`, `experimentos.py`, `actualizar_con_llm_real.py` y `notebooks/{01_corpus, 02_modelos_clasicos, 03_hibrido}.ipynb`. Toda la generación es reproducible con semilla fija (`RANDOM_STATE=42`); la re-ejecución del LLM requiere una clave de API de OpenRouter provista por variable de entorno (no incluida en el repositorio).

Repositorio: <https://github.com/Rodvdev/cs3061-intent-whatsapp>

REFERENCIAS

- [1] H. Chen, X. Liu, D. Yin, and J. Tang, “A survey on dialogue systems: Recent advances and new frontiers,” *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 2, pp. 25–35, 2017.
- [2] G. Tur and R. De Mori, *Spoken Language Understanding: Systems for Extracting Semantic Information from Speech*. Hoboken, NJ, USA: John Wiley & Sons, 2011.
- [3] S. Larson, A. Mahendran, J. J. Peper, C. Clarke *et al.*, “An evaluation dataset for intent classification and out-of-scope prediction,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 1311–1316.
- [4] A. Coucke, A. Saade, A. Ball, T. Bluche *et al.*, “Snips voice platform: An embedded spoken language understanding system for private-by-design voice interfaces,” *arXiv preprint arXiv:1805.10190*, 2018.
- [5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

- [6] B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, 2008.
- [7] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Proc. 10th European Conf. on Machine Learning (ECML)*. Springer, 1998, pp. 137–142.
- [8] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proc. 15th Conf. of the European Chapter of the ACL (EACL)*, Vol. 2, 2017, pp. 427–431.
- [9] X. Liu, A. Eshghi, P. Swietojanski, and V. Rieser, "Benchmarking natural language understanding services for building conversational agents," in *Proc. 10th Int. Workshop on Spoken Dialogue Systems (IWSDS)*, 2019, pp. 165–183.
- [10] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [12] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [13] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson, and I. Vulić, "Efficient intent detection with dual sentence encoders," in *Proc. 2nd Workshop on NLP for Conversational AI (NLP4ConvAI)*, 2020, pp. 38–45.
- [14] J.-G. Zhang, K. Hashimoto, W. Liu *et al.*, "Discriminative nearest neighbor few-shot intent detection by transferring natural language inference," in *Proc. EMNLP*, 2020, pp. 5064–5082.
- [15] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [16] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. EMNLP*, 2014, pp. 1532–1543.
- [17] A. Vaswani, N. Shazeer, N. Parmar *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [20] L. Ouyang, J. Wu, X. Jiang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 730–27 744.
- [21] A. Conneau, K. Khandelwal, N. Goyal *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annual Meeting of the ACL*, 2020, pp. 8440–8451.
- [22] S. Schuster, S. Gupta, R. Shah, and M. Lewis, "Cross-lingual transfer learning for multilingual task oriented dialog," in *Proc. NAACL-HLT*, 2019, pp. 3795–3805.
- [23] J. FitzGerald, C. Hench, C. Peris *et al.*, "MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages," in *Proc. 61st Annual Meeting of the ACL*, 2023, pp. 4277–4302.
- [24] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 6382–6388.
- [25] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.